

day2_hw1_report

과제 1

Qnode(qnod.hpp/ qnode.cpp) : ROS2통신 전달. QThread를 상속 해서 별도 스레드에서 rclcpp::spin_some을 반복해서 실행한다.

- publishVelocity(linear, angular) : /turtle1/cmd_vel 에 Twist 메세지 발생, 발행할 때마다 cmdVelUpdated signal도 같이 emit한다.

- setPen(r, g, b, width, off) : turtle1/set_pen 서비스 비동기 호출

- /turtle1/pose 구도/ -> 위치가 갱신될 때마다 poseUpdate(x, y, theta) signal emit한다.

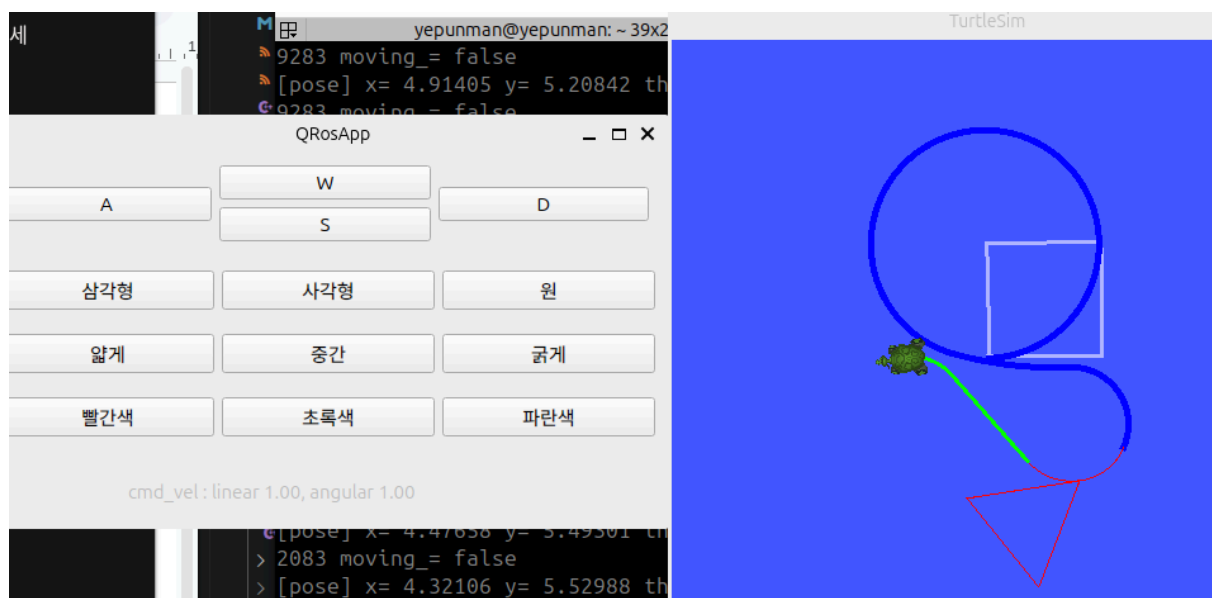
MainWindow (main_window.hpp/ main_window.cpp) : GUI 로직, 버튼 클릭과 키보드 입력을 받아 QNode의 함수를 호출하고, QNode가 보내는 signal을 받아 화면을 갱신한다.

발생한 시행착오

처음에 도형을 그릴때 시간 기반으로 구현했는데 사각형을 그리다가 거북이가 덜 움직이거나 회전이 정확하지 않아 모서리가 90도가되지 않는 문제가 발생했다. 이 문제의 원인은 오차 가 누적되어서 발생하는 문제였다. 이를 해결하기 위해 pose피드백 기반 페루프 제어를 구현하기로 했다.

거북이가 움직이는 과정에서 디버깅하기 위해 로그를 출력하게 만들었는데 로그를 보니 로그 이후 진행 코드가 전혀 찍히지 않는 문제가 있었다. 원인을 조사해보니 turtlesim은 약 1초간 새cmd_vel을 받지 못하면 자동으로 속도를 0으로 리셋하는 안전장치 때문이었다. 거북이가 회전할 때 1초 이상 걸리게 되는데 이때 속도가 0으로 초기화 되어 거북이가 회전 이후 움직이지 않았던 것이다. 이 문제는 pose콜백이 올 때마다 같은 속도 명령을 계속 재발행하도록 코드를 수정하여 해결하였다.

회전 각도가 약간 더크게 나오는 문제가 있었다. 위치 확인 주기가 너무 길어서 발생한 문제였는데 이 문제는 위치확인 주기를 줄이고, 회전 목표지점(90도, 120)근처에서 회전 속도를 줄여서 해결했다.



실행 결과를 보면 흰색(기본색) 네모, 굵은 파란색 원, 얇은 세모를 확인 할 수 있다. 그리고 초록색 중간 굵기 선은 wasd로 제어한 거북이의 경로를 나타낸다. 그리고 ui아래 cmd_vel을 보면 linear = 1.0, angular = 1.0으로 a키로 좌회전하고 있는 상황인 것을 알 수 있다.